

Unidade III

5 FUSÃO DO PRODUTO E DO PROCESSO DE SOFTWARE

A fusão do software com o processo de desenvolvimento para gerar artefatos ou sistemas de software ajuda as empresas desenvolvedoras a consolidar suas operações, reduzindo os custos e aumentando a produtividade.

Essa fusão representa a integração estreita entre a definição do produto final e as atividades necessárias para desenvolvê-lo. É uma abordagem holística que assegura que os requisitos do software e seus processos de desenvolvimento sejam considerados simultaneamente.

O software é o resultado final do desenvolvimento, incluindo elementos como código, interfaces do usuário e do hardware, documentação, funcionalidades e desempenho esperado.

O processo de software abrange as camadas da engenharia de software, como qualidade, processos, métodos, ferramentas e técnicas utilizadas para criá-lo. Inclui desde a engenharia de requisitos, arquiteturas, codificação, testes unitários e de integração até a manutenção contínua.

A prática das metodologias ágeis e as práticas DevOps promovem a colaboração entre equipes de desenvolvimento e operações. São utilizadas ferramentas de integração e entregas contínuas fundamentais para automatizar e monitorar o progresso do projeto, de forma que o produto final atenda aos padrões de qualidade desejados.

De acordo com Sommerville (2011), as quatro atividades básicas do processo – especificação, desenvolvimento, validação e evolução – são organizadas por processos que diferem conforme o produto a ser gerado e os métodos aplicados pela equipe de desenvolvimento. Por exemplo: ao utilizar o modelo de processo Cascata, as atividades são organizadas de forma sequencial; já no modelo Incremental, as iterações, revisões sucessivas do processo, permitem mudanças nas fases do ciclo de desenvolvimento.

As atividades dependem do tipo de software a ser elaborado, dos desenvolvedores e da estrutura organizacional. Por exemplo: na metodologia ágil Scrum, as especificações e as tarefas são escritas em cartões (Kanban).

As principais características de um bom processo são:

- planejamento adequado com objetivos e cronograma claros;
- gestão eficaz de requisitos;

- modelagem e arquiteturas com base em padrões do tipo UML;
- iterações que permitem ciclos curtos e melhorias contínuas;
- garantia da qualidade com revisões, testes e diagnósticos;
- documentação detalhada e atualizada;
- gestão de mudanças para se adaptar às necessidades do cliente, às mudanças no ambiente operacional e a possíveis falhas no projeto;
- riscos identificáveis e mitigados;
- comunicação e colaboração entre as partes envolvidas no projeto;
- ferramentas para integração de uso repetitivo para automatizar os processos e os métodos.

O bom processo deve ser bem definido, configurável, adaptável e repetível, com nomenclatura e métricas padronizadas entre clientes e desenvolvedores.

5.1 Domínio da análise do engenheiro de software

Já tratamos das características pessoais do engenheiro de software, agora acentuaremos seu domínio de análise, que se refere ao conjunto de áreas de conhecimento e aspectos técnicos, organizacionais e de negócio que esse profissional deve considerar para desenvolver soluções eficazes alinhadas às necessidades dos usuários, da equipe e da organização.

Essa análise envolve a compreensão detalhada do problema ou das necessidades a serem implementadas, o conhecimento de normas e padrões da qualidade do software, processos organizacionais, de desenvolvimento e tecnologias envolvidas, ferramentas, infraestrutura de TI e comunicação com clientes e usuários.

A engenharia de software é uma tecnologia em camadas. Como ilustra a figura 36, qualquer abordagem (inclusive engenharia de software) deve estar fundamentada em um comprometimento organizacional com a qualidade (Pressman, 2021).

A visão da tecnologia em camadas fornece uma base sólida de conhecimentos diversos, que mostra a dimensão de uma boa parte das habilidades profissionais necessárias ao engenheiro de software.

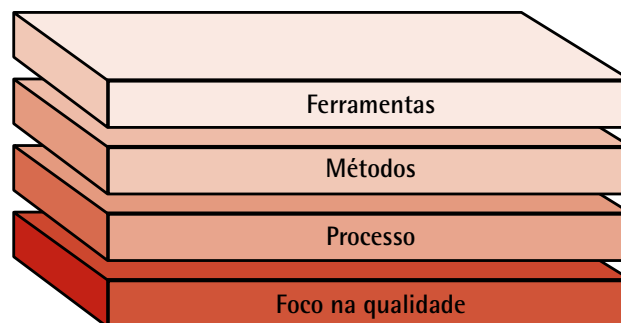


Figura 36 – Camadas da engenharia de software

Adaptada de: Pressman (2021).



Lembrete

O software é o resultado final do desenvolvimento, incluindo elementos como código, interfaces do usuário e do hardware, documentação, funcionalidades e desempenho esperado.

5.1.1 Foco na qualidade

A qualidade estabelece regras, padrões, normas, procedimentos e atividades que orientam o desenvolvimento de software de alta qualidade, levando à cultura de melhoria contínua dos processos e a abordagens cada vez mais eficientes da engenharia de software.

Ela assegura que o produto final atenda à expectativa dos usuários. No entanto, se o usuário não estiver satisfeito, toda a excelência técnica torna-se irrelevante e nada mais realmente importa.



Satisfação do usuário = produto adequado
+ máxima qualidade
+ entrega dentro do orçamento e do cronograma

Figura 37

Observe a seguir alguns exemplos de normas.

- ISO/IEC 25010 (SQuaRE – Systems and Software Quality Requirements and Evaluation – Requisitos de Qualidade e Avaliação de Sistemas e Software):
 - Define os modelos de qualidade para sistemas e software, incluindo características de qualidade e subcaracterísticas como funcionalidade, desempenho, segurança e manutenibilidade.
 - Substitui a antiga norma ISO/IEC 9126 e a ISO/IEC 14598.

- **ISO 12207 – Processos do ciclo de vida do software:**
 - Padroniza os processos envolvidos no ciclo de vida do software, desde a concepção até a manutenção.
- **ISO 15504 – SPICE:**
 - Define um modelo para avaliação e melhoria de processos de software, permitindo medir a maturidade dos processos organizacionais.
 - Serve de base para modelos como o CMMI.
- **ISO 9001 – Sistema de gestão da qualidade:**
 - Foco na padronização e melhoria contínua. A ISO 9001 é referência na garantia de gestão da qualidade em projeto, instalação, desenvolvimento, produção, arquitetura e serviço.

Agora observe alguns modelos.

- **CMMI:** os níveis de maturidade acentuados a seguir representam diferentes estágios de evolução dos processos de uma organização. Esses níveis ajudam a identificar a capacidade de uma organização em desenvolver processos eficazes e eficientes.
 - Modelo de maturidade que avalia e melhora processos de desenvolvimento de software.
 - Usado por empresas para garantir eficiência e previsibilidade no desenvolvimento.
 - Tem cinco níveis de maturidade:
 - **Inicial (initial):** processos não padronizados.
 - **Gerenciado (managed):** processos controlados.
 - **Definido (defined):** processos padronizados.
 - **Quantitativamente gerenciado (quantitatively managed):** controle por métricas.
 - **Em otimização (optimizing):** melhoria contínua.
- **MPS.BR (Melhoria do Processo de Software Brasileiro)**
 - Desenvolvido em 2003 pela Softex (Associação para Promoção da Excelência do Software Brasileiro), com o apoio do MCTI (Ministério da Ciência, Tecnologia e Inovações).

- Elaborado para atender micro, pequenas e médias empresas, fazendo uso das principais abordagens internacionais voltadas para a definição, a avaliação e a melhoria dos processos de software.
- Compatível com o CMMI, ele tem sete níveis de maturidade alinhados aos cinco níveis do CMMI.
- Os sete níveis de maturidade no modelo MPS.BR são:
 - G (parcialmente gerenciado).
 - F (gerenciado).
 - E (parcialmente definido).
 - D (largamente definido).
 - C (definido).
 - B (gerenciado quantitativamente).
 - A (em otimização).
- **ITIL (Information Technology Infrastructure Library – Biblioteca de Infraestrutura de Tecnologia da Informação):** é o conjunto de melhores práticas para gerenciamento de serviços de TI, incluindo suporte e manutenção de software.
- **COBIT (Control Objectives for Information and Related Technologies – Objetivos de Controle para Informação e Tecnologias Relacionadas):** trata-se de modelo de governança de TI que ajuda empresas a alinhar software e tecnologia aos objetivos estratégicos do negócio.

5.1.2 Processo

Define a estrutura fundamental para a concepção e o desenvolvimento de software orientando-se por padrões e normas da qualidade estabelecida. O processo assegura a integração das camadas tecnológicas, permitindo um desenvolvimento de software racional e oportuno.

O processo estabelece uma metodologia a ser seguida para a entrega eficaz de tecnologia, constituindo a base para o controle e o gerenciamento do projeto. Ele pode ser identificado quando o analista especifica uma série organizada de eventos e atividades, acompanhada de documentação e modelos que orientam a obtenção de resultados específicos, seja na prestação de um serviço, seja na construção de um produto.

Um framework do ciclo de desenvolvimento de software é composto basicamente pelos processos:

- **Elicitação (início ou concepção):** compreende o gerenciamento das relações com o cliente, a comunicação e a interpretação do negócio com o objetivo de levantar os requisitos do software. A intenção é identificar as entidades e as operações e compreender os objetivos dos stakeholders (conjunto dos interessados).
- **Análise:** após a interpretação do negócio, elabora-se um modelo conceitual do plano de projeto de software, que abrange a especificação detalhada do software a ser desenvolvido. Esse plano contempla a definição das atividades e tarefas técnicas, a estruturação do cronograma, a gestão de riscos, além da alocação de recursos e a estimativa de custos envolvidos no processo. Faz-se nessa etapa a relação entre as necessidades funcionais do cliente e a capacidade da equipe ou empresa desenvolvedora para construir o software.
- **Especificação e modelagem:** a especificação define com clareza e precisão as características, os requisitos e as condições de algo. Normalmente, ela é acompanhada de um modelo gráfico com a modelagem, que expressa a especificação. Essa ação envolve detalhar as funcionalidades, os comportamentos e as restrições de um sistema ou componente, fornecendo uma descrição completa que orienta a sua construção, implementação e verificação.
- **Construção:** é o processo de codificação (geração do código), depuração (exame e limpeza dos dados, do código e da interface), testes unitários e diagnósticos.
- **Implementação:** é a integração do código construído ao software, realizando-se nessa etapa testes de integração e diagnósticos.
- **Implantação:** corresponde à fase de entrega do release do software ao cliente, que, por sua vez, fornece feedbacks de avaliação para validar e aceitar o software.
- **Validação:** é o processo que envolve a garantia da qualidade ao cliente. Esse acordo entre cliente e desenvolvedor garante que o software produzido atende às necessidades e expectativas do cliente ou do usuário final. É a confirmação de que o produto faz o que deve fazer de maneira correta e adequada para seu propósito.

Esses frameworks são mostrados nos modelos de processos:

- **PSP:** criado pelo SEI, é específico para engenheiros de software.
- **TSP:** também criado pelo SEI, seu foco é a equipe de trabalho.
- **Cascata:** modelo clássico do ciclo de vida de desenvolvimento do software.
- **Prototipagem:** modelo iterativo com atividades para montagem de protótipos.

- **Incremental:** modelo iterativo desenvolvido com o conceito de versões.
- **RAD:** usando frameworks de linguagens de programação, visa à entrega rápida de sistemas de software, adotando uma estratégia baseada na construção modular e orientada a componentes reutilizáveis.
- **Espiral:** modelo evolucionário que combina a natureza iterativa da prototipagem com os aspectos sistemáticos do Cascata.
- **RUP:** processo proprietário de engenharia de software criado pela Rational Software Corporation e atualmente representado pela IBM.

5.1.3 Métodos

É o conjunto estruturado de procedimentos, regras e técnicas utilizados para guiar o desenvolvimento do software para alcançar uma meta, um fim ou um conhecimento. Um método assegura que as etapas do ciclo de vida do software sejam seguidas de maneira sistemática e eficiente.

O método determina os passos a serem seguidos na construção do software ou parte dele. Envolve um amplo conjunto de tarefas, desde gestão de equipes, análise de requisitos, projeto e construção de programas, até os testes, a entrega, o suporte e a manutenção.

Os métodos são princípios fundamentais que regem cada área da tecnologia, englobando atividades de modelagem e outras técnicas descritivas que garantem a precisão e a eficiência no desenvolvimento de soluções tecnológicas.

Serão acentuados alguns exemplos de métodos.

Metodologias ágeis

Trata-se de uma coleção de metodologias baseada na prática para modelagem efetiva de sistemas de software e podem ser aplicadas por profissionais no dia a dia.

Em contraponto aos modelos de processos estruturados, as metodologias ágeis são específicas para cada atividade do ciclo de desenvolvimento e, basicamente, são aplicadas em projetos orientados a objetos, podendo ser integradas em determinadas atividades dos modelos de processos estruturados.

A metodologia ágil que mais se destaca é o Scrum, por ser flexível e adaptável, com entregas incrementais, foco no cliente e melhoria contínua. Ele pode ser integrado a outras metodologias ágeis, como XP, Kanban, FDD e demais metodologias ágeis que se fizerem necessárias no desenvolvimento do software.

MR (Matriz de Responsabilidades)

É um método de gestão para definir papéis e responsabilidades de cada membro da equipe em relação às atividades e tarefas do projeto, com o objetivo de deixar claro quem está envolvido e qual função deve desempenhar.

A MR traz melhoria nas comunicações e no controle das atividades, podendo se associar a técnicas de cronogramas para distribuir com detalhes a tarefa de cada membro da equipe e técnicas de construção de diagramas de rede de tarefas (ou rede Kanban), a exemplo de PERT (Program Evaluation and Review Technique – Técnica de Revisão e Controle de Programas) e CPM (Critical Path Method – Método do Caminho Crítico).

A MR é orientada pelo PMBOK na gestão de stakeholders, tema que será tratado mais adiante.

PERT

Preocupa-se com o fator tempo, com o objetivo de planejar e controlar o projeto. Os prazos são tratados de forma probabilística, indicando que, inicialmente, os períodos são estimados com base em experiências anteriores.

CPM

Preocupa-se com o fator custo, em que são aplicadas técnicas de planejamento e controle para realização das tarefas por meio de cálculos determinísticos, o que significa que, pela CMP ou rede de cartões (Kanban), é possível ajustar os períodos das tarefas em um período total mais curto.

Outros métodos

Amplamente constantes na engenharia de software, pode-se citar os métodos de:

- engenharia de requisitos;
- modelagem e design;
- implementação e programação;
- testes de software;
- manutenção e evolução de software.



Lembrete

O método determina os passos a serem seguidos na construção do software ou parte dele. Envolve um amplo conjunto de tarefas, desde gestão de equipes, análise de requisitos, projeto e construção de programas, até os testes, a entrega, o suporte e a manutenção.

5.1.4 Ferramentas

Trata-se de recursos tecnológicos que dão apoio automatizado para os processos e os métodos em todas as fases do desenvolvimento de software.

As ferramentas de software agilizam as tarefas, aumentam a produtividade, a comunicação e a colaboração entre as equipes. Entre elas, pode-se citar ferramentas para gestão de projetos, modelagem, IDE, controle de versões, inspeção da qualidade do código, testes, documentação etc.

Citaremos a seguir alguns exemplos de ferramentas de software.

Gestão de projetos

Na análise de customização dos serviços, o software GanttProject, fornecido gratuitamente, tem várias funcionalidades compatíveis com o Project, da Microsoft, que é pago.

Com o GanttProject é possível obter cronogramas, gráfico de análise do caminho crítico da rede de tarefas e diagrama de recursos (ou pessoas), entre outras aplicações. Esses gráficos e análises são os resultados da automação dos métodos PERT/CPM, MR e Kanban.

O levantamento desses dados é feito por um especialista em gestão de projetos de software, sendo obtidos por meio da análise de dados referente aos métodos utilizados e requisitos do software.

A tabela 1 mostra a aplicação dos métodos PERT/CPM, como técnica de revisão de controle da programação da sprint, referente à metodologia ágil Scrum.

A tabela PERT combina as tarefas da sprint (e backlogs) a serem realizadas em uma ordem lógica, considerando o tempo esperado (te) em duração de dias para realizar a tarefa e a relação com as tarefas anteriores.

Tabela 1 – Tabela PERT da lista de tarefas do Scrum

Item	Tarefa	Tarefas anteriores	Duração (dias)
0	Planejamento da sprint (sprint planning)	–	5
1	Desenvolvimento do incremento (sprint execution)	0	10
2	Plano de reuniões diárias (daily scrum)	0	1
3	Revisão da sprint (sprint review)	0, 1, 2	3
4	Retrospectiva da sprint (sprint retrospective)	3	1
5	Refinamento do backlog (backlog refinement)	4	2
6	Plano para entrega incremental	5	2
7	Documentação técnica	4, 5, 6	2
8	Plano para suporte técnico	6, 7	1

A figura 38 ilustra a implementação dos dados da tabela PERT com a utilização do GanttProject: como resultado, é obtido o cronograma do projeto. Na parte esquerda da figura tem-se a lista de tarefas, equivalente ao "input dos dados" da tabela PERT. Na parte direita, acentua-se o cronograma gerado automaticamente.

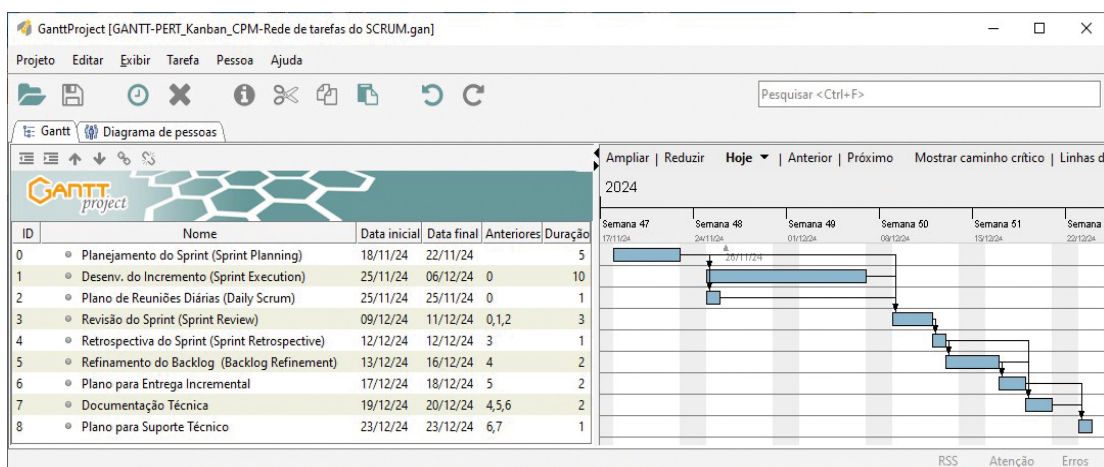


Figura 38 – Tela do software GanttProject com o cronograma gerado

Por sua vez, a figura 39 mostra o gráfico PERT como resultado obtido pelo GanttProject, referente aos dados da tabela PERT. Esse gráfico apresenta um Kanban. Com ele, pode-se fazer o CPM entre as tarefas, possibilitando a identificação de melhorias para reduzir os períodos necessários à execução de cada tarefa.

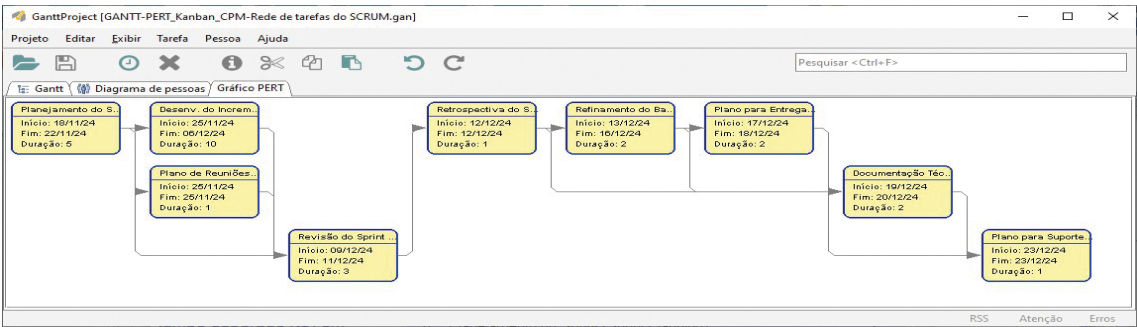


Figura 39 – Tela do software GanttProject com o cronograma gerado

O mercado de software disponibiliza uma ampla variedade de ferramentas para a gestão de projetos. No quadro 5 estão descritas algumas das principais ferramentas de gestão de projetos.

Quadro 5

Ferramenta	Especificação
GanttProject	Ferramenta de gestão de projetos, de código aberto para pequenas e médias empresas, bem como para equipes ágeis. Permite criar gráficos de Gantt, gerenciamento de recursos e cálculos de custos de projetos
Git	Sistema de controle de versão distribuído, é utilizado para rastrear alterações no código-fonte. Plataformas como GitHub e GitLab fornecem interfaces adicionais e funcionalidades colaborativas
Jira	Desenvolvido pela Atlassian, é uma ferramenta para a gestão de projetos ágeis. Suporta Scrum, Kanban e métodos híbridos, permitindo o rastreamento de tarefas, bugs e histórias de usuário
Microsoft Project	Ferramenta poderosa para planejamento, agendamento e controle de projetos. Suporta gráficos de Gantt e outros recursos avançados de gestão de projetos
Microsoft Teams	Solução para comunicação e colaboração em tempo real, com funcionalidades de chat, videochamadas e integração com as ferramentas do Office 365
Trello	Desenvolvido pela Atlassian, usa um sistema de quadros e cartões para gerenciar tarefas e projetos. É altamente visual e fácil de usar, ideal para equipes que preferem uma abordagem simplificada

Com essas ferramentas, é possível criar modelos gráficos detalhados do modelo de negócio, da aplicação, dos dados e da infraestrutura de TI, os quais acompanham as especificações de requisitos do software. Essas ferramentas facilitam a visualização e o design de sistemas complexos, permitindo uma melhor compreensão e comunicação entre os membros da equipe de desenvolvimento.

Como exemplo, a figura 40 mostra o uso da ferramenta de modelagem Visio para a construção de um diagrama de sequência pelo padrão UML. O diagrama de sequência representa a interação entre os componentes do sistema ao longo do tempo, especificando claramente como são realizados os processos e as operações.

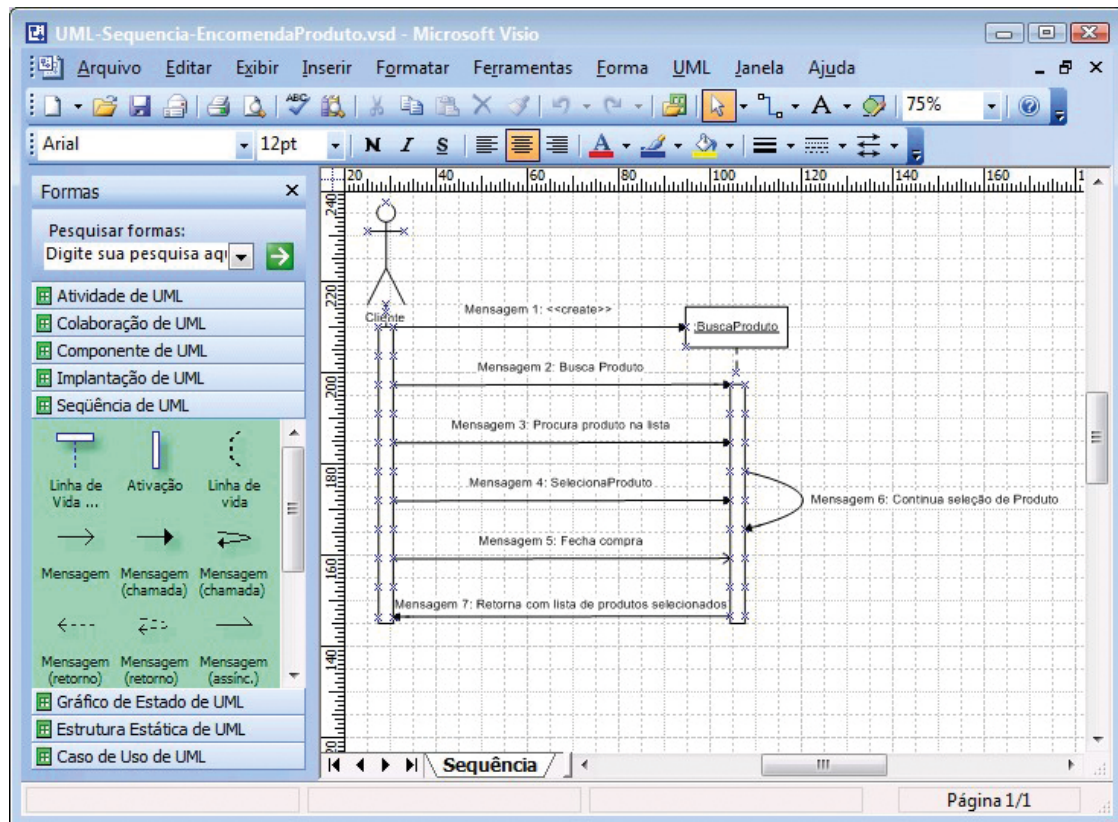


Figura 40 – Modelagem de software com diagrama de seqüências elaborado pela ferramenta Visio, da Microsoft

O mercado de software disponibiliza uma gama de aplicações pagas e gratuitas como ferramentas de software para modelagem e o quadro 6 acentua algumas das principais.

Quadro 6

Ferramenta	Especificação
Astah	Permite criar diagramas UML de UC, atividades, classes, seqüência, componentes, implantação, entre outros. Tem recursos de exportação/importação de arquivos ou programas e integração com ferramentas de desenvolvimento
Bizagi	Utilizada na BPM, com suporte para criação de diagramas BPMN e automatização de processos empresariais
Draw.io	É uma ferramenta de modelagem e diagramação. Permite criar uma variedade de diagramas, desde fluxogramas, diagramas UML, design de telas de software, mapas mentais, diagramas em rede etc.
FreeMind	É uma ferramenta de modelagem mental que permite aos usuários criar mapas mentais para organizar, visualizar informações, criar mapas de navegação em software, entre outros. O mapa mental é utilizado em abordagens para levantamento de requisitos e recursos, brainstorm, planejamento e organização de ideias
MySQL Workbench	É uma ferramenta de design de banco de dados que permite criar e visualizar esquemas de banco de dados, realizar engenharia direta e reversa e gerenciar bases de dados MySQL
Microsoft Visio	É uma ferramenta de modelagem e diagramação. Com gráficos vetoriais, foi lançada pela primeira vez em 1992 pela Visio Corporation e a Microsoft adquiriu a empresa em 2000. É usada para criar uma variedade de diagramas, incluindo fluxogramas, organogramas, planos de piso, diagramas de rede, diagramas UML e mapas mentais

IDE

É um framework de programação que opera no domínio da aplicação, fornecendo uma solução completa para uma "família de problemas" específicos de uma determinada funcionalidade. Essencialmente, é composto por um conjunto de classes e interfaces, decomposto em quadros com seus respectivos códigos.

Os frameworks acentuados permitem a reutilização de componentes de software, padronizam a arquitetura do sistema e promovem a consistência no desenvolvimento.

A figura 41 traz o exemplo de uso do framework de programação NetBeans, que está sendo usado para o desenvolvimento de classes. Observe as caixas de comandos na tela: são caixas de acesso a bibliotecas e ferramentas integradas para agilizar o processo de codificação e garantir a integridade do código.

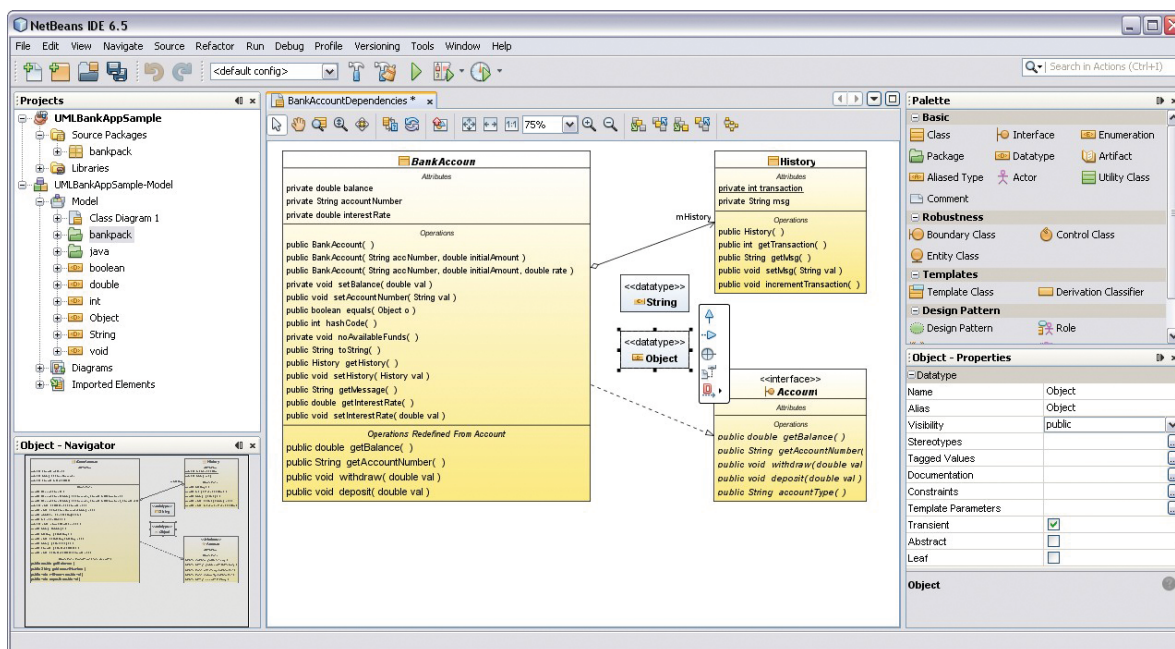


Figura 41 – Ambiente de trabalho na codificação e modelagem de software com o framework de programação NetBeans

Existem diversas ferramentas IDE para o desenvolvimento de software e o quadro 7 a seguir destaca algumas delas.

Quadro 7 – Ferramentas IDE

Ferramenta	Especificação
ASP.NET Core (C#)	De código aberto, é uma ferramenta para a construção de aplicações web modernas baseadas em.NET
Django (Python)	Promove o desenvolvimento rápido e o design limpo e pragmático. É ideal para construir aplicações web robustas
Eclipse	Desenvolvido pela Eclipse Foundation, é um dos IDEs mais populares e versáteis disponíveis atualmente, operando com diversas linguagens de programação em ambiente multiplataforma
Electron (JavaScript, HTML, CSS)	Permite a criação de aplicações desktop multiplataforma usando tecnologias web
NetBeans	Plataforma de desenvolvimento de software modular, foi inicialmente desenvolvida pela Sun Microsystems e hoje é mantida pela Apache Software Foundation
Qt (C++)	Ferramenta robusta e completa para o desenvolvimento de aplicações desktop e móveis com suporte multiplataforma
React Native (JavaScript)	Ferramenta para desenvolvimento de aplicativos móveis nativos usando React, permite a criação de apps para iOS e Android a partir de um único código-fonte
Xamarin (C#)	Utiliza o.NET para construir aplicativos móveis para Android e iOS com uma única base de código compartilhada

5.2 JAD (Joint Application Development – Desenvolvimento de Aplicação Conjunta)

De acordo com Fournie (1994), o JAD é uma metodologia criada pela IBM em 1977 e tem por finalidade reunir uma equipe de usuários, stakeholders e desenvolvedores para alcançar acordos baseados na compreensão das operações do processo.

O JAD é uma metodologia colaborativa de ciclo rápido utilizada no desenvolvimento de software para acelerar o processo de comunicação, por meio de reuniões estruturadas denominadas sessões JAD. Essas sessões são lideradas por um facilitador, que orienta a discussão e garante que as partes interessadas contribuam com suas perspectivas para determinar requisitos, design de software e resolução de problemas.

A JAD reúne usuários finais e especialistas em sistemas de informação em uma sessão interativa para discutir o projeto do sistema. Se adequadamente preparadas e mediadas, as sessões JAD podem acelerar significativamente a fase de projeto e envolver intensamente os usuários (Laudon; Laudon, 2014, p. 416).

As sessões JAD, conforme mostra a figura 42, incluem representantes das diversas áreas do conhecimento, com o objetivo principal de reunir as partes interessadas no projeto para definir requisitos de software e especificações.



Figura 42 – Sessão JAD. Imagem gerada por IA por meio da OPENAI

Disponível em: <https://shre.ink/eB08>. Acesso em: 23 jun. 2025.

Vamos acentuar a seguir os perfis dos principais participantes.

Facilitador (ou líder da sessão)

Conduz a sessão JAD. É responsável pelas interações entre os participantes, contribuindo para que os objetivos sejam alcançados.

O líder deve garantir que o processo revisional prossiga conforme o planejado, controlar as interações entre os participantes e arbitrar quando necessário.

Esse profissional deve dominar o relacionamento interpessoal e ser experiente na condução efetiva de reuniões. É respeitado tecnicamente, independente e tem envolvimento no projeto.

Executivo patrocinador (autor do produto, analista de negócios ou analista de processos)

É a representação da gerência de mais alto nível, pertencente às fileiras dos usuários, estando muito comprometido com as atividades do processo de negócio.

Esses profissionais fornecem as diretrizes sobre as principais metas e objetivos do negócio. Muitas vezes, eles são o próprio cliente patrocinador do projeto.

Gerência funcional (ou representante do usuário)

Trata-se das pessoas peritas na matéria em discussão nas sessões JAD, são aquelas que precisam do software. Elas devem ser escolhidas pela sua capacitação profissional em descrever as práticas atuais e operações do negócio que serão implementadas no software a ser desenvolvido.

Gerente de projeto

Auxilia na coordenação da sessão JAD com o objetivo de alinhar os interesses estratégicos da organização com os recursos necessários para produzir o software.

Esse profissional atua na gestão da equipe de desenvolvimento e stakeholders com a preocupação de controlar a comunicação, os custos e o tempo do projeto. Elabora cronogramas, planilhas de custo e distribuição dos recursos de uma forma geral.

Gerente de projeto da infraestrutura de TI

Atua diretamente com a equipe de desenvolvimento, avaliando a capacitação do pessoal de desenvolvimento para produzir o software e os recursos de infraestrutura.

Esse profissional atua na gestão da equipe específica do desenvolvimento. Elabora arquiteturas, distribuição dos recursos de TI e faz análise de Kanban para melhoria da produtividade.

Analistas de sistemas (e/ou desenvolvedores)

São aqueles que trabalham no desenvolvimento do software, que levam conhecimentos técnicos e específicos de como os assuntos de negócios serão implementados no software.

Esses profissionais documentam os requisitos e ajudam na interpretação das necessidades dos usuários em especificações técnicas, analisam a viabilidade técnica das soluções propostas e identificam possíveis riscos.

Durante as sessões JAD, os analistas (com o secretário ou não) elaboram protótipos do software para que usuários e clientes possam ver o que se pretende ser mostrado nele.

Secretário (escriba ou documentador)

Registra apenas as informações relevantes das discussões e das decisões tomadas durante a sessão. Para tal, usa ferramentas de software para organizar, documentar e apontar tendências dos assuntos pertinentes na discussão. O secretário JAD é muitas vezes citado como um especialista no uso de ferramentas CASE e de prototipação.

5.3 Gerenciamento da equipe de desenvolvimento

A gestão de equipes de desenvolvimento de software assegura que todas as partes interessadas estejam devidamente informadas e engajadas em uma comunicação eficaz e melhor alocação de recursos.

Para promover um ambiente colaborativo e adaptável que atenda às necessidades de stakeholders e equipe do projeto, a metodologia sugerida baseia-se na MR do PMBOK. Também conhecida como MR RACI, ajuda a definir claramente papéis e responsabilidades de cada participante da equipe de

projeto. Atende a todo e qualquer projeto que necessite alinhar os recursos humanos com as tarefas a serem realizadas, como é o caso da formação de equipes de desenvolvimento nos modelos de processos tradicionais e metodologias ágeis.

As principais aplicações da MR na engenharia de software envolvem:

- **Definição clara de papéis e responsabilidades:** com ela, todos os membros da equipe de stakeholders sabem exatamente suas funções e responsabilidades no projeto, o que melhora a comunicação e evita sobreposições e redundâncias de tarefas.
- **Gestão de recursos:** a MR ajuda na alocação e no controle eficiente dos recursos, garantindo que o indivíduo trabalhe especificamente na sua tarefa e saiba também as responsabilidades dos demais companheiros de equipe que trabalham na atividade, facilitando o relacionamento das partes e a hierarquia de decisão na realização da atividade.
- **Integração com os métodos PERT/CPM:** são métodos de gerenciamento de projetos aplicados ao desenvolvimento e à produção em diversos ramos da indústria, neste caso, a indústria de software.
- **Integração com metodologias ágeis:** nos ambientes de desenvolvimento ágil como XP, Scrum, FDD e outras, as responsabilidades podem mudar de sprint para sprint.

A transparência proporcionada pela integração da MR com metodologias ágeis é útil em projetos de software, pois, neles, as mudanças são frequentes e a colaboração contínua é essencial, promovendo uma melhor gestão dos recursos e dos prazos. Essa integração permite que a equipe tenha uma estrutura flexível e adaptável de forma rápida, melhorando a produtividade de software com Scrum, por exemplo.

5.3.1 Análise de dados e construção da MR

Na construção da MR neste livro-texto, são considerados stakeholders todos os interessados no software com a seguinte hierarquia de decisão: cliente, representante do usuário, autor do processo, gerente de projeto, gerente do sistema, programador, DBA (Data Base Administrator – Administrador do Banco de Dados).



Observação

O conceito dos termos stakeholders e desenvolvedores é usado com bom senso. Na aplicação do método MR, vale a própria definição de stakeholder: qualquer indivíduo, grupo ou organização que tem interesse direto ou indireto em um projeto, produto ou serviço e pode afetar ou ser afetado pelo resultado.

Dessa forma, nos exemplos a seguir serão considerados todos os participantes do projeto, ou seja, estão inclusos representantes, com perfil administrativo, outros interessados no projeto e desenvolvedores.

Uma matriz é uma tabela de números, símbolos ou expressões, organizados em um quadro retangular que comporta linhas e colunas, em que cada elemento (ou célula) da matriz é identificado por dois índices: um para a linha e outro para a coluna, conforme o quadro 8.

Matriz de responsabilidades (MR). Uma tabela que mostra os recursos do projeto alocados a cada pacote de trabalho. A matriz RACI é uma maneira comum de mostrar às partes interessadas quem são responsáveis, as pessoas que devem prestar contas, ser consultadas ou informadas, associadas às atividades, decisões e entregas do projeto (PMI, 2021, p. 240).

O quadro 8 mostra uma MR para a gestão de equipe de desenvolvimento. De acordo com os participantes e as atividades, essa matriz se refere à fase do processo de concepção do software.

Quadro 8 – Distribuição de colunas e linhas de uma MR para a gestão da equipe de desenvolvimento

ID	Atividade	Stakeholder				
		Cliente	Gerente de projeto	Gerente de sistemas	Engenheiro de software	Analista de sistemas
1	Elicitação					
2	Análise de requisitos					
3	Modelagem do negócio					

Veja onde são distribuídos os atributos RACI no quadro 9. Observe que é atribuído um R apenas para cada atividade e que normalmente o atributo R é acompanhado pelo atributo C, que pode ser distribuído o quanto você achar necessário.

Quadro 9 – MR RACI para a gestão da equipe de desenvolvimento no processo de concepção do software

ID	Atividade	Stakeholder				
		Cliente	Gerente de projeto	Gerente de sistemas	Engenheiro de software	Analista de sistemas
1	Elicitação	C	R	C	I	–
2	Análise de requisitos	C	A	R	C	I
3	Modelagem do negócio	A	C	C	–	R

A distribuição dos atributos RACI não é atribuída ao acaso. A identificação das responsabilidades é vital para o controle do recurso humano no processo organizacional.

Para determinar os critérios de responsabilidades, algumas questões devem ser observadas quanto aos integrantes da equipe de desenvolvimento:

- Qual é o organograma que será aplicado para os stakeholders?
- Quem é o responsável mais cobrado por um determinado resultado: do processo, da atividade ou da tarefa?
- Quem influencia mais no processo e tem maior poder de decisão?
- Quem é o autor do processo?
- Quem faz a maior parte do trabalho ou quais são os integrantes que são mais cobrados no processo?

A distribuição dos atributos deve se orientar de acordo com o grau de envolvimento do integrante da equipe na atividade. Na MR RACI, a definição e a função para cada atributo do processo na montagem da matriz são acentuadas a seguir:

- **R (Responsável):** é o integrante responsável pelo processo, atividade ou tarefa. Ele alinha a equipe, cria procedimentos de serviços e dispõe os recursos necessários para a execução da atividade. É quem responde pelo sucesso de execução da atividade. Para todas as atividades, sem exceção, deve existir obrigatoriamente apenas um R.
- **A (Aprova):** é o integrante responsável pela validação (ou aceite) dos resultados da atividade, que normalmente é o cliente (interno ou externo), o autor da atividade, a diretoria ou a gerência. Deve ser usado apenas um atributo A para a atividade, porém não existe obrigatoriedade para aplicá-lo em todas as atividades, porque nem todas elas necessitam de aprovação.
- **C (Consultado):** envolve os auxiliares diretos que trabalham na atividade junto ao responsável (R). Os integrantes que dão suporte podem representar um determinado membro da equipe, terceiros contratados para trabalhar na atividade ou consultores. Para cada R é sugerido que seja acompanhado por pelo menos um C.
- **I (Informado):** engloba os convidados a integrar a equipe para a realização de uma determinada atividade. Os informados atuam na atividade de forma indireta e podem representar vários perfis da equipe. Eles podem ter o perfil de consultores que têm conhecimentos específicos sobre uma determinada atividade ou interessados na atividade por alguma razão, por exemplo, avaliar o custo da atividade quando esta envolver as operações de verificação de integração dos componentes do software.

Exemplo de aplicação

Caso: gestão de equipe de desenvolvimento Scrum com MR RACI.

Antes de iniciar as tarefas da metodologia ágil Scrum, deve-se fazer o alinhamento das tarefas com os serviços a serem desenvolvidos pela(s) equipe(s) Scrum. Isso é feito uma única vez e vale para a(s) equipe(s) que faz(em) parte do desenvolvimento do software em questão.

A distribuição das tarefas está relacionada às responsabilidades de cada participante da equipe e, com a MR RACI, se define o papel do participante no projeto.

Tarefas do Scrum

Criação do backlog do produto (product backlog), uma lista priorizada dos itens e dos requisitos do produto que é criada e mantida pelo product owner.

- **Planejamento da sprint (sprint planning):** é a seleção dos itens do backlog do produto que definem o objetivo da sprint, a qual é realizada pelo product owner com a equipe de desenvolvimento.
- **Execução da sprint (sprint execution):** é a execução do backlog do produto pela equipe de desenvolvimento.
- **Reuniões diárias (daily stand-ups):** são reuniões rápidas (15 minutos) que ocorrem com a participação de cada membro da equipe.
- **Revisão da sprint (sprint review):** é a revisão do trabalho concluído com a demonstração dos incrementos do produto. Ela ocorre ao término da sprint com a equipe de desenvolvimento, o product owner e as partes interessadas.
- **Retrospectiva da sprint (sprint retrospective):** é a análise da sprint para verificar se tudo está funcionando bem. Para tal, a equipe realiza testes, diagnósticos e sugestões de melhoria no produto.
- **Incremento do produto (product increment):** ao término do trabalho, a sprint é integrada ao software, implantada e condicionada à validação do cliente. Nessa fase, participam todos da equipe e os interessados no produto.
- **Stakeholders:** são considerados aqui os participantes que formam a estrutura do desenvolvimento do software com o Scrum.
- **Cliente:** não tem um papel definido no Scrum, porém participa do desenvolvimento fornecendo feedbacks do produto.
- **Mentor ágil (agile coach):** orienta e treina a equipe quanto a princípios e práticas ágeis.

- **Proprietário do produto (product owner):** é um analista de sistema que prioriza os backlogs mais importantes da sprint.
- **Mestre do Scrum (Scrum master):** faz o papel na equipe do gerente do sistema, garantindo que a equipe siga as práticas do Scrum. Ele facilita o processo e remove impedimentos que possam atrapalhar a evolução do desenvolvimento.
- **Desenvolvedor (developer):** é responsável pela codificação, pela implementação e pelos testes dos requisitos, de forma a garantir que o produto esteja funcionando de acordo com o esperado.

Construção da MR RACI: distribuição dos atributos RACI na MR

Apesar de considerarmos um modelo básico e estático de sete tarefas (linhas) e cinco stakeholders (colunas), a distribuição dos atributos da matriz é dinâmica. Não existe especificamente uma distribuição estática dos atributos, esses parâmetros são variáveis, porque tudo depende do projeto, do custo, do prazo, do interesse na troca de conhecimento e na melhoria da comunicação.

O analista responsável pela construção da matriz deve ter um bom senso crítico para a definição da quantidade de tarefas, de stakeholders e de distribuição dos atributos. O objetivo é ter um workflow eficiente.

A matriz apresentada no quadro 10 é um modelo aplicado a um caso específico e que pode ser usado em projetos similares.

Quadro 10 – MR RACI do Guia PMBOK para gestão de equipe Scrum

ID	Tarefa	Stakeholders				
		Cliente	Agile coach	Product owner	Scrum master	Desenvolvedor
1	Criação do backlog do produto	A	–	R	C	–
2	Planejamento da sprint	I	I	R	C	C
3	Execução da sprint	–	C	C	A	R
4	Reuniões diárias	–	I	C	R	C
5	Revisão da sprint	A	C	R	C	C
6	Retrospectiva da sprint	–	–	I	R	C
7	Incremento do produto	I	I	A	R	C

A aplicação da matriz não para por aí! Com o uso de uma ferramenta de software que construa gráficos de Gantt, é possível integrar na matriz os métodos PERT/CPM para criar cronogramas, diagrama de Kanban, efetuar análises de caminho crítico na rede de cartões e análise na distribuição de períodos e custos para cada tarefa ou horas acumuladas de serviços pelo stakeholder, além da customização completa do desenvolvimento do software.



Observação

Uma boa estratégia para a gestão de equipes de desenvolvimento de software é o comprometimento para atender a demanda sem sobrecarga de serviços.

6 PROCESSO UNIFICADO

Trata-se de um framework de desenvolvimento de software que oferece uma abordagem iterativa e incremental. Ele enfatiza a importância da arquitetura, do gerenciamento de riscos e da qualidade do software.

O processo unificado tem como base o RUP, a implementação comercial mais conhecida. Entre as suas diversas ramificações, destacam-se:

- **Praxis:** é um framework para desenvolvimento de software que se baseia nos princípios do processo unificado, mas com foco em sistemas embarcados e de tempo real. Ele observa a importância da modelagem formal e da verificação rigorosa para garantir a confiabilidade e a segurança de sistemas críticos.
- **Iconix:** é uma abordagem de desenvolvimento de software ágil que simplifica o processo unificado, tornando-o mais leve e adaptável a projetos menores e equipes com menos experiência. O foco do Iconix é se concentrar na modelagem visual e na geração automática de código, para agilizar o desenvolvimento, mantendo os princípios essenciais do processo unificado.
- **AUP (Agile Unified Process – Processo Unificado Ágil):** mantém as principais fases do RUP (concepção, elaboração, construção e transição), porém com iterações mais curtas e flexíveis. É uma versão simplificada e adaptada para metodologias ágeis com ênfase em colaboração, feedbacks rápidos e entrega contínua de software funcional. Também envolve o RUP-Scrum Hybrid.
- **EUP (Enterprise Unified Process – Processo Unificado Empresarial):** é uma extensão do RUP que visa tratar de todo o ciclo de vida do software, não se limitando apenas ao desenvolvimento, incorporando as fases de produção e desativação do software.
- **OpenUP (Open Unified Process – Processo Unificado Aberto):** leve e flexível, é projetado para ser adaptado a diferentes tipos de projetos e equipes. O OpenUP busca simplicidade, colaboração e entrega de valor ao cliente.

6.1 RUP: fases e workflow

Trata-se de um processo de engenharia de software. Originalmente desenvolvido pela Rational Software Corporation, agora está integrado à IBM, como IRUP (IBM Rational Unified Process).

O RUP oferece uma estrutura disciplinada para a produção e a atribuição de software de alta qualidade, com foco em práticas recomendadas e em uma abordagem iterativa e incremental. É distribuído online, utilizando tecnologia web, disponibilizando upgrades regulares de software pela Rational Software. A figura 43 mostra o ambiente operacional de desenvolvimento via web.

Ele é configurado conforme as necessidades de cada organização. É documentado, desenhado, desenvolvido, distribuído e mantido como uma ferramenta de software usando a UML.

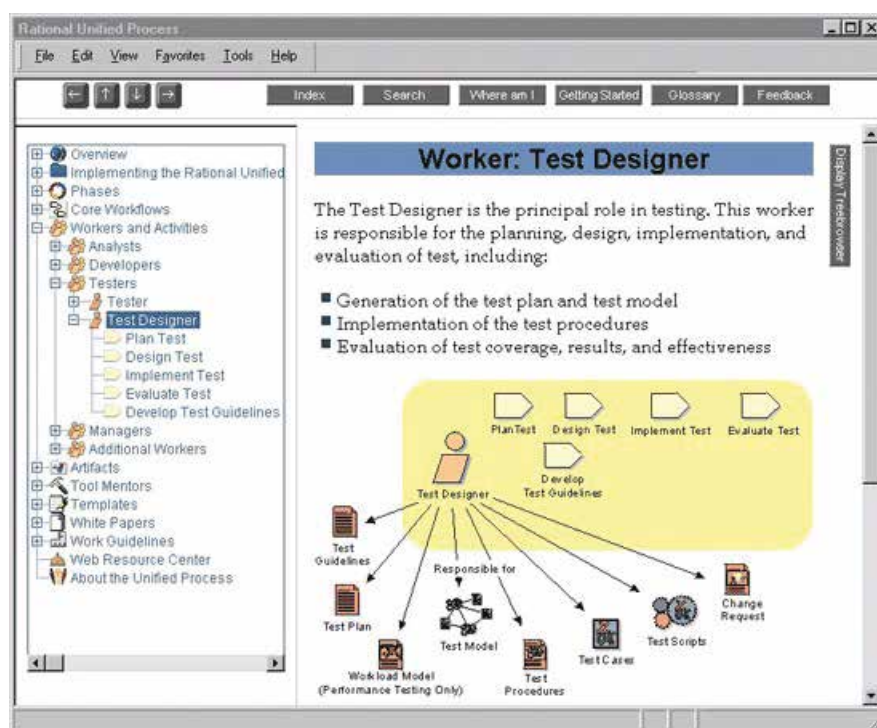


Figura 43 – Plataforma de trabalho RUP pela web

Fonte: Kruchten (2000, p. 19).

O RUP oferece uma abordagem baseada em disciplinas para atribuir tarefas e responsabilidades dentro de uma organização de desenvolvimento. Sua meta é garantir que a produção de software de alta qualidade atenda às necessidades dos usuários dentro do cronograma e com orçamentos previsíveis. Ele captura as principais boas práticas modernas da engenharia de software: o desenvolvimento iterativo, a gestão de requisitos, a arquitetura baseada em componentes, o uso de software de modelos visuais, a verificação contínua da qualidade e gestão e o controle de mudanças de software.

Vamos acentuar a seguir as principais características do RUP.

Desenvolvimento iterativo e incremental

No desenvolvimento de um software de média à alta complexidade, não é possível definir um problema e uma solução eficiente em uma única etapa.

Os requisitos mudam com frequência, devido a vários fatores como:

- restrições nas arquiteturas da lógica de programação, da informação e da infraestrutura de TI;
- mudanças nas necessidades primárias do cliente e mudanças ocorridas devido a um refinamento do problema inicialmente levantado.

Cada iteração resulta em um incremento do software com novas funcionalidades e refinamentos, correspondendo a um novo release. As iterações permitem reduzir o risco do projeto e o acompanhamento do progresso.

Modelagem orientada a objetos (OO)

Usa a UML para modelagem OO do software, o que facilita a compreensão e a comunicação entre os membros da equipe e outras partes interessadas.

A linguagem UML se transformou em um padrão para as indústrias ao representar projetos e é usada amplamente pelo RUP (Kruchten, 2000).

Ao representar o projeto utilizando construções gráficas, o RUP expressa de forma eficiente uma visão geral da solução, o que permite que clientes sem nenhuma afinidade com a área de desenvolvimento de sistemas possam facilmente compreender o projeto e adquirir mais conhecimentos sobre a evolução do desenvolvimento do projeto.

Arquitetura baseada em componentes

O RUP é baseado na arquitetura de componentes desde o início do projeto. Ela é desenvolvida e validada nas fases iniciais, servindo como pilar para o desenvolvimento subsequente.

Uma arquitetura baseada em componentes viabiliza um sistema extensível, promovendo a reutilização de "peças" do software. Promove a definição inicial de uma arquitetura de software robusta, que facilita a parametrização do desenvolvimento, a reutilização e a manutenção.

O RUP suporta essa sistemática de construção de sistema, concentrando-se em uma arquitetura executável nas primeiras fases do projeto (Kruchten, 2000).

Centrado em UC

O modelo RUP é conduzido por UC, observe o framework da figura 44. Os UC capturam os RS e dirigem o processo de desenvolvimento.

O RUP descreve como documentar as funcionalidades, as restrições do sistema, as restrições do projeto e os requisitos através do UC. Isso ocorre desde a análise dos requisitos até testes do sistema finalizado para implantação.

A partir do UC são criadas diversas visões sustentadas por outros diagramas da UML, que compõem uma determinada funcionalidade, por exemplo: visão da lógica, do processo, do desenvolvimento e a visão da implementação. Isso ajuda a garantir que o software atenda às necessidades dos usuários.

Os UC e os cenários são exemplos de artefatos do processo que se mostram altamente eficazes para documentar os requisitos funcionais (Kruchten, 2000).

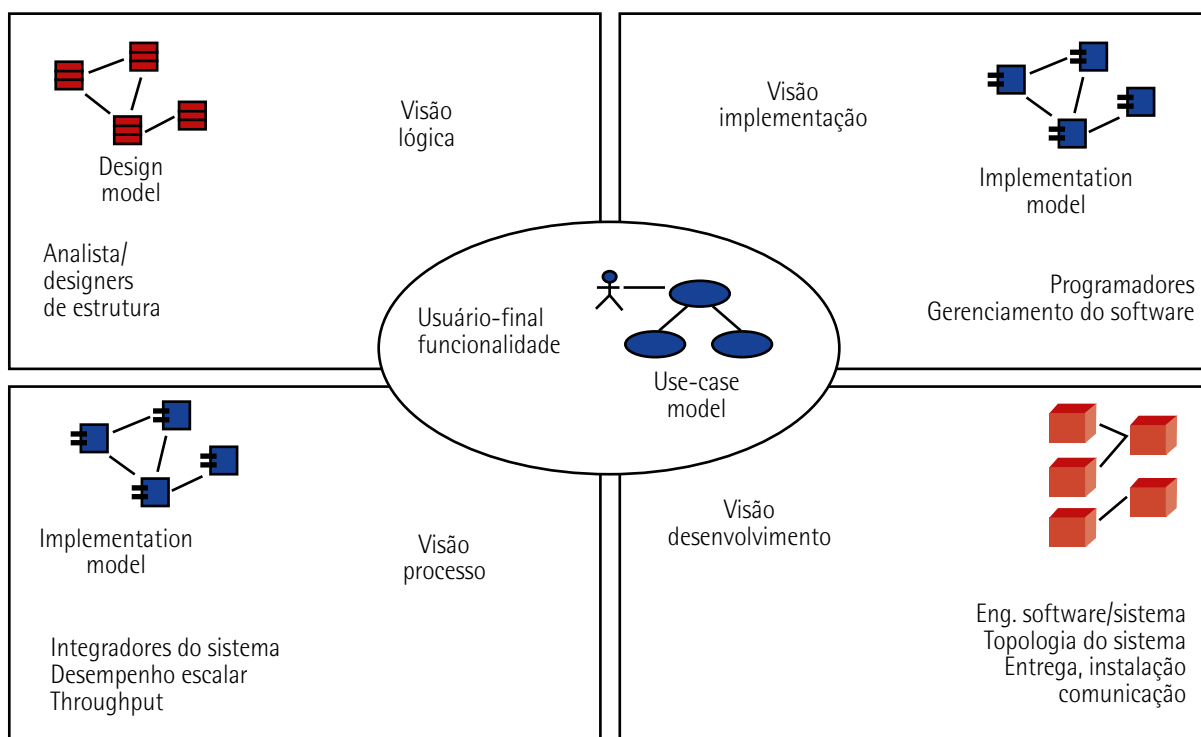


Figura 44 – Construção de funcionalidade a partir do UC, de acordo com o RUP

Adaptada de: Kruchten (2000).

Gerenciamento de riscos

Identifica e gerencia riscos ao longo do ciclo de vida do desenvolvimento do projeto, permitindo que a equipe tome medidas proativas para mitigá-los.

Fases bem definidas e disciplinas estruturadas

O framework do RUP divide o processo em fases, formando a estrutura dinâmica, que acompanha todo o ciclo de desenvolvimento, bem como a estrutura estática com as disciplinas (workflows). Elas variam com a evolução do ciclo de desenvolvimento, proporcionando uma abordagem estruturada e organizada.

Documentação

Enfatiza a criação e a manutenção de documentação abrangente e detalhada ao longo do desenvolvimento, facilitando a manutenção e a evolução do sistema.

Customizável

Pode ser adaptado para atender às necessidades específicas de diferentes projetos e organizações, oferecendo flexibilidade na implementação.

Verificação contínua da qualidade

Assegurar a qualidade do software durante o desenvolvimento do projeto é algo intensivo feito pelo RUP, que assiste todo o processo envolvendo todos os membros no controle e no planejamento da qualidade (Kruchten, 2000).

Gestão e controle de mudanças do software

O RUP define métodos para controlar e monitorar as mudanças a fim de garantir que mudanças não comprometam inteiramente o sucesso do projeto (Kruchten, 2000).

6.1.1 Framework do RUP

Ilustrado na figura 45, apresenta um processo bidimensional, mostrando o ciclo de vida do desenvolvimento de software praticado no RUP.

O framework é dividido em fases e cada ciclo resulta em uma nova geração do produto ou parte dele. Cada fase é dividida por iterações definidas para cada projeto.

O RUP apresenta um modelo gráfico com dois eixos: o horizontal tem quatro fases; o vertical, seis disciplinas de trabalho e três disciplinas gerenciais.

Na dimensão horizontal estão as fases do ciclo de vida do desenvolvimento, designadas como estrutura dinâmica, que representa as mudanças ao longo da linha de tempo que ocorrem no processo de desenvolvimento.

Na dimensão vertical estão as disciplinas de trabalho (ou workflow), denominadas como estrutura estática, expressando as mudanças no esforço despendido no desenvolvimento.

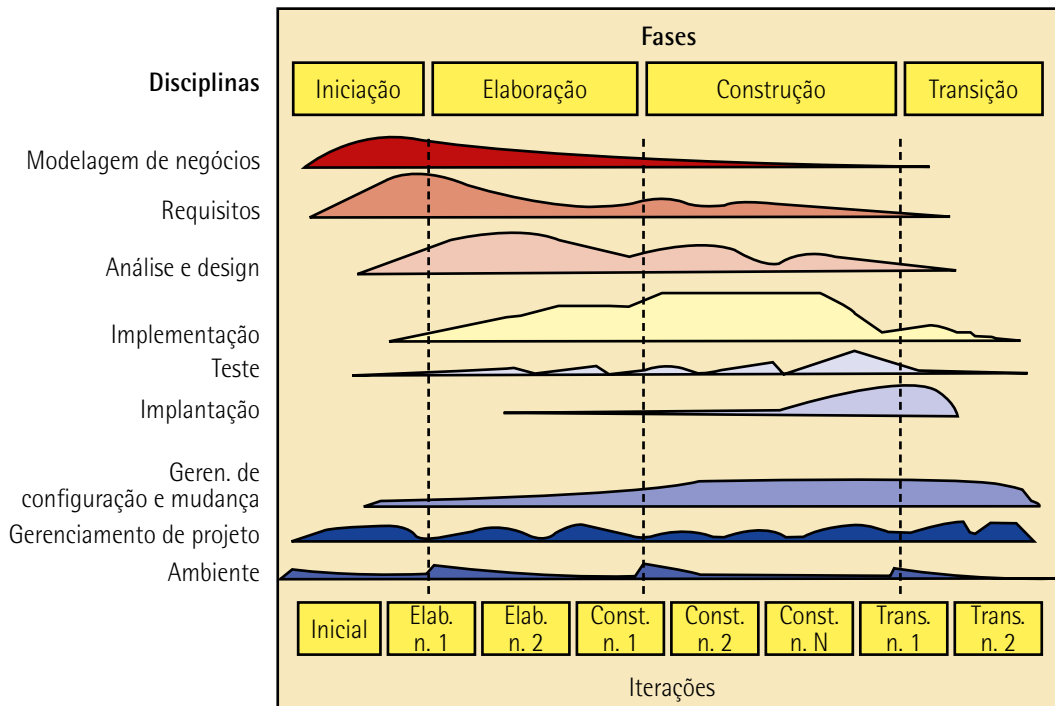


Figura 45 – Framework do RUP com suas fases e disciplinas (workflows)

Adaptada de: Kruchten (2000).

6.1.2 Estrutura dinâmica

Acompanhada de diversas iterações, ela envolve as quatro fases do RUP ao longo do ciclo de vida de desenvolvimento do software. Ao término de cada uma das fases, faz-se uma breve retrospectiva do processo e prepara-se a próxima etapa, verificando se os objetivos da fase atual foram concluídos. Após completar as quatro fases, uma versão release do software estará completa. Para cada nova versão, o software passa por todas as fases novamente (desenvolvimento evolucionário).

Acentuaremos a seguir as características das fases do RUP que acompanham o ciclo de desenvolvimento:

- **Fase 1: Iniciação/Concepção (Inception/Conception)**

- Definir o escopo e as fronteiras do sistema (contrato com o cliente).
- Elaborar um UC crítico ao sistema (atores e requisitos).
- Criar o business case do sistema, que deve incluir os critérios de sucesso, a avaliação de riscos, a estimativa de recursos e o cronograma do ciclo de desenvolvimento.

- Ao término da fase, deverá haver a concordância de stakeholders e desenvolvedores, a compreensão dos requisitos e a credibilidade nas estimativas. O projeto pode ser cancelado ou fortemente reformulado se falhar nesse aspecto.

- **Fase 2: Elaboração (Elaboration)**

- Garantir que o projeto, a arquitetura e o conjunto de requisitos estejam estáveis para garantir o cumprimento de prazos e custos.
- Produzir protótipos evolucionários e exploratórios.
- Eliminar os elementos de maior risco.
- Descrever requisitos adicionais (não funcionais e com menor grau de importância).
- Revisar o business case.
- Ao término da fase, deve-se obter estabilidade da arquitetura, demonstrar que o projeto é viável (tempo e custo) e que ele pode ser cancelado ou reformulado se falhar nesse aspecto.

- **Fase 3: Construção (Construction)**

- Minimizar custos por meio da utilização ótima de recursos.
- Desenvolver versões utilizáveis.
- Completar os processos de análise, projeto, implementação e testes das funcionalidades do sistema.
- Desenvolver de maneira iterativa e incremental um produto que esteja pronto para ser entregue aos usuários, por meio de um manual e de uma descrição da versão corrente.
- Decidir se o software e os usuários estão prontos para a implantação do sistema.
- Ao término da fase, deve-se verificar se a disponibilização pode ser adiada se os critérios não forem cumpridos.

- **Fase 4: Transição/Implantação (Transition) – atividades de entrega do software**

- Realizar testes de integração para validar o novo sistema.
- Treinar usuários e pessoas responsáveis pela manutenção.
- Iniciar as tarefas de marketing e distribuição.
- Ao término da fase, deve-se obter a satisfação dos clientes e os gastos devem ser aceitáveis.

6.1.3 Estrutura estática

Na estrutura estática, o RUP é representado por nove disciplinas (seis de trabalho e três gerenciais). Cada uma delas utiliza quatro elementos primários de modelagem: papéis, atividades, artefatos e workflows.

- **Papéis**

- Descrevem o comportamento e as responsabilidades de um indivíduo ou grupo de indivíduos de uma equipe.
- O comportamento de cada papel é dado pelo conjunto de atividades desempenhadas por ele.
- As responsabilidades de cada papel estão associadas a artefatos que devem ser desenvolvidos ao longo do processo.
- Exemplos de papéis incluem especificador de UC, arquiteto de softwares, projetista de interface e gerente de projetos.

- **Atividades**

- Realizadas pelos papéis, consistem em ações que atualizam ou geram artefatos.
- A granularidade de uma atividade é expressa em horas ou dias.
- As atividades são divididas em etapas de tarefas: entendimento, realização e revisão.

Quadro 11 – Modelo para a distribuição de papéis e respectivas atividades

Papel	Atividade
Gerente de projetos	Planejamento das iterações
Gerente de sistemas	Especificação e modelagem
Analista de sistemas	Determina UC e atores
Analista de testes de desempenho	Executa testes de desempenho

- **Artefatos**

- Informações produzidas, modificadas ou utilizadas ao longo do desenvolvimento de software.
- São utilizados como entrada para atividades e são produzidos como saída.
- Exemplos incluem modelos (UC, projetos), documentos (plano de negócios, plano de projeto, conjunto de artefatos, de componentes e outros) e código-fonte.

- **Workflows**

- Definem uma sequência de atividades que produzem resultados observáveis na forma de artefatos.
- A modelagem pode ser expressa em forma de diagramas de atividades, sequência e colaboração.
- Como mostra o quadro 12, o workflow do RUP é apresentado em dois grupos, compostos por seis disciplinas de trabalho e três gerenciais.

Quadro 12 – Workflow do RUP

Disciplinas de trabalho	Especificação
Modelagem de negócio (business modeling)	A intenção é melhorar o entendimento do negócio através do desenvolvimento de um modelo de negócios. Os documentos gerados são UC de negócio e modelo de objetos de negócio
Requisitos (requirements)	Possibilita um melhor entendimento dos RS partindo de um acordo com o cliente e com objetivo de oferecer uma orientação aos desenvolvedores. São produzidos um modelo de UC detalhado e um protótipo do sistema
Análise e projeto (analysis and design)	Os requisitos capturados na disciplina anterior são transformados em projeto. Modelos de análise e projeto são gerados
Implementação (implementation)	Nesta fase, o projeto é transformado em código. A estratégia é desenvolver o sistema em camadas, particionando em subsistemas. O resultado final é um componente testado que faz parte do produto final
Teste (test)	Elaboração de testes e verificação se todos os requisitos foram cumpridos. Os documentos gerados são os modelos e casos de testes
Utilização (deployment)	Torna o produto disponível para o usuário final. Responsável pelo empacotamento do produto, instalação, treinamento ao usuário e distribuição do produto
Gerenciamento de configuração	Controla as modificações e mantém a integridade dos artefatos do projeto
Gerenciamento de projeto	Descreve várias estratégias para o trabalho com um processo iterativo
Ambiente	Abrange infraestrutura necessária para o desenvolvimento do sistema

6.2 Extensões do RUP

6.2.1 Praxis

De acordo com Paula Filho (2015), o modelo Praxis foi desenvolvido pela equipe de Barry Boehm, fundamentando-se no modelo de ciclo de vida Espiral, com as mesmas raízes no RUP, porém com estrutura diferente das disciplinas organizacionais.

Trata-se de uma metodologia que combina teoria e prática de maneira dinâmica, muitas vezes associada a metodologias ágeis.

Igualmente ao RUP, o Praxis é dirigido por UC, centrado na arquitetura, iterativo e incremental. A modelagem do Praxis é feita com uso do EPF (Eclipse Process Framework), que facilita a adaptação, a personalização, a extensão e a evolução. Herda a arquitetura RUP, complementada por um metamodelo da parte estática do RUP e pelo perfil exclusivo do Praxis.

Ele também se conecta aos métodos ágeis, como Scrum, Kanban e XP; enfatizando a aplicação prática das ideias teóricas em um ciclo dinâmico, promovendo aprimoramento contínuo entre prática e teoria.

Observe o framework do método Praxis na figura 46. Ele é uma abordagem integrada que une diferentes aspectos da gestão de projetos, programas e portfólios; permitindo que organizações e indivíduos desenvolvam maturidade, ampliem o conhecimento e aprimorem competências de maneira contínua e alinhada. Os quatro pilares são: métodos, competências, enciclopédia e comunidade.

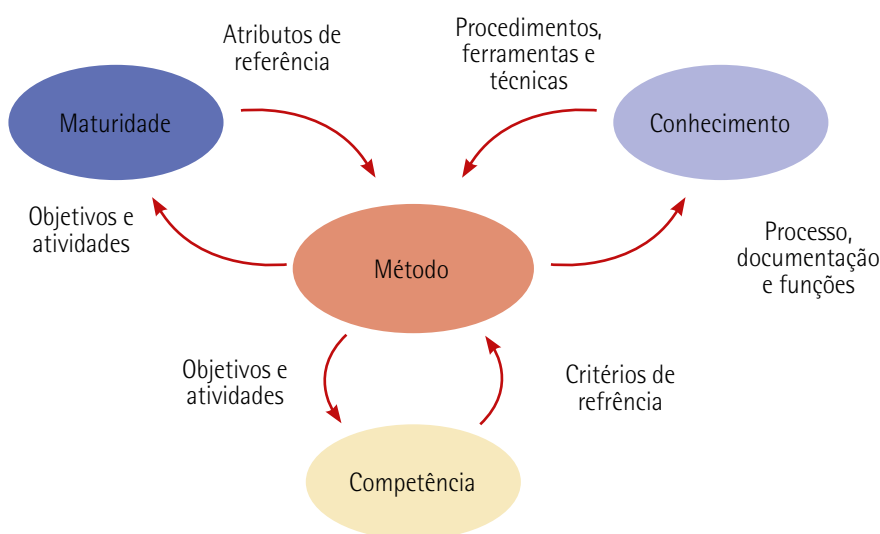


Figura 46 – Framework do método Praxis



Saiba mais

Para entender melhor essa metodologia, consulte:

PRAXIS. *Visão geral*. 2023. Disponível em: <https://shre.ink/egKn>. Acesso em: 23 jun. 2025.

6.2.2 Iconix

O processo Iconix é uma metodologia proprietária de desenvolvimento de software baseada na UML. O processo é orientado a caso de uso e usa diagramas baseados em UML para definir quatro marcos. A principal característica do processo é um conceito chamado modelagem de robustez,

baseado no trabalho inicial de Ivar Jacobson, que ajuda a preencher a lacuna entre análise e design (Iconix, s.d.).

No desenvolvimento de software, a metodologia Iconix antecede tanto o RUP, a XP e as práticas de desenvolvimento ágil.

Os modelos comportamentais apresentados na figura 47 são construídos com a ferramenta Enterprise Architect. Ela compila os modelos a partir de diagramas específicos como máquina de estado, sequência, atividades e outros modelos da UML para gerar código na linguagem descritiva do software. As linguagens suportadas incluem C(OO), C++, C#, Java, VB.Net, VHDL, Verilog e SystemC.

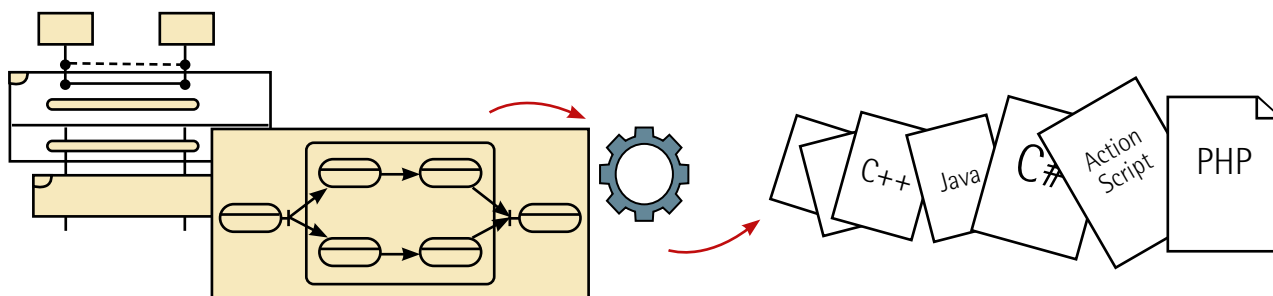


Figura 47 – Modelos comportamentais baseados em diagramas da UML são gerados pelo Enterprise Architect na automatização da codificação

Os quatro marcos principais do Iconix que ajudam a estruturar o processo de desenvolvimento são:

- **Modelagem do domínio:** inicialmente, a partir do levantamento de requisitos, é criado o modelo de domínio para identificar os conceitos-chave do sistema, utilizando diagramas UML, como diagramas de classes, para representar os objetos e suas relações.
- **Revisão de requisitos:** são extraídos UC para especificar os RF do sistema. Esse marco é importante para que o design seja alinhado às expectativas e às necessidades do cliente.
- **Revisão preliminar do design:** dos UC são extraídos diagramas de sequência e de colaboração, convertendo os UC em interações entre objetos, conectando requisitos ao design técnico do software.
- **Revisão crítica do design:** é a validação dos diagramas de design que garante que o modelo esteja completo e que o sistema pode ser implementado com base nos artefatos criados.

6.2.3 EUP

De acordo com Ambler (2023), o EUP estende processos iterativos/ágeis, como DAD (Disciplined Agile Delivery), XP ou Scrum para atender às necessidades reais das organizações de TI.



Saiba mais

O EUP é uma extensão do RUP que visa tratar de todo o ciclo de vida do software, não se limitando apenas ao desenvolvimento, incorporando no ciclo de vida as fases Production (Produção) e Retirement (Desativação), como pode ser observado no gráfico exposto na indicação a seguir:

ENTERPRISE Unified Process (EUP): strategies for enterprise Agile. *Ambsoft Inc*, [s.d.]. Disponível em: <https://shre.ink/xEpP>. Acesso em: 23 jun. 2025.

6.2.4 OpenUP

É um processo unificado de código aberto, leve e flexível, projetado para ser adaptado a diferentes tipos de projetos e equipes; busca simplicidade, colaboração e entrega de valor ao cliente.

O OpenUP é uma metodologia de desenvolvimento de software que faz parte do EPF. Criado como uma versão mais leve e ágil do RUP, tem basicamente o mesmo ciclo de vida estruturado do RUP, com as quatro fases principais: concepção, elaboração, construção e transição; mantendo suas características essenciais, mas simplificando e reduzindo a complexidade. É ideal para equipes pequenas, podendo ser adaptado para diferentes contextos e necessidades, enfatizando a agilidade e a colaboração entre desenvolvedores e stakeholders.

Existem dois objetivos do EPF:

- Fornecer uma estrutura extensível e ferramentas exemplares para engenharia de processos de software para criação de métodos e processos, gerenciamento de bibliotecas, configuração e publicação do processo.
- Fornecer conteúdo de processo exemplar e extensível para uma variedade de processos de desenvolvimento e gerenciamento de software que suportem desenvolvimento iterativo, ágil e incremental, sendo aplicável a um amplo conjunto de plataformas e aplicativos de desenvolvimento (Eclipse..., s.d.).

6.3 Métodos, ferramentas e técnicas: definição de papéis e responsabilidades para atividades do RUP

No contexto do RUP, vimos que a matriz MR RACI é uma excelente ferramenta para organizar e comunicar as atribuições individuais e coletivas das equipes envolvidas.

A definição de papéis e responsabilidades com MR RACI garante clareza e eficácia no gerenciamento de projetos de software, permitindo categorizar as responsabilidades de forma estruturada e padronizada, associando cada atividade do RUP às principais funções, como analista de negócios, gerente de projeto, arquiteto de software, gerente do sistema, analista dos sistemas, desenvolvedor e demais stakeholders.

A abordagem da matriz MR RACI, conforme as diretrizes do RUP, assegura que cada fase do ciclo de vida do desenvolvimento esteja alinhada às funções definidas, aumenta a colaboração entre os membros da equipe, reduz ambiguidades e promove um fluxo de trabalho mais eficiente.

O modelo apresentado na figura 48 exemplifica essa aplicação. Considerando as fases e o workflow (disciplinas) do RUP para a montagem da MR RACI e as dimensões do RUP, nessa aplicação serão ponderadas as disciplinas do RUP como as atividades do processo a serem desempenhadas. As responsabilidades atribuídas a stakeholders e desenvolvedores serão acentuadas nas fases do ciclo de vida do RUP.

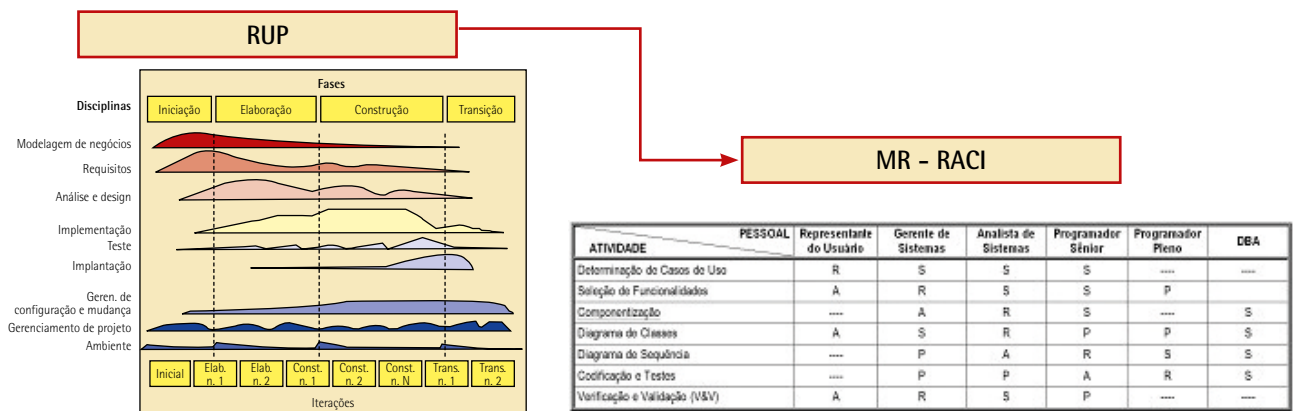


Figura 48 – Modelo de aplicação da MR RACI no processo unificado

Acompanhe a montagem da matriz observando a figura 45. Para a construção da MR RACI, observe o quadro a seguir e suas especificações.

Quadro 13 – Matriz de distribuição de papéis e responsabilidades para o processo unificado

ID	Disciplinas	Stakeholders e desenvolvedores								
		CL	US	AN	GP	GS	AS	PR	TE	DB
1	Modelagem de negócios	A	C	R	C	C	I	–	–	I
2	Requisitos	C	C	A	C	R	C	I	–	I
3	Análise e design	C	C	C	A	R	C	I	I	I
4	Implementação	I	I	C	I	C	C	R	C	C
5	Teste	I	C	I	I	A	C	C	R	C
6	Implantação	C	A	C	R	C	C	C	C	C
7	Gerenciamento de configuração e mudança	–	–	–	C	R	C	C	C	C
8	Gerenciamento de projeto	A	C	C	R	C	C	C	C	C
9	Ambiente	C	C	C	C	A	R	C	I	I
Fases		Inicição		Elaboração		Construção			Transição	
Atributos da matriz RACI: R – Responsável; A – quem Aprova; C – Consultado (suporte); I – Informado (participa)										
Cliente (CL), Usuário (US), Analista de Negócios (AN), Gerente de Projeto (GP), Gerente de Sistema (GS), Analista de Sistemas (AS), Programação (PR), Tester (TE) e Administrador do Banco de Dados (DB)										

Análise dos dados da MR RACI

Uma parte da análise é atribuir uma pontuação para os atributos de acordo com a hierarquia de decisão. Por exemplo: R = 5 pontos; A = 4 pontos; C = 3 pontos; e I = 2 pontos. Essa pontuação pode ser convertida posteriormente em valores de tempo ou custo.

Observe a fase de Construção na figura 45:

- **Acumulado em GS:** R = 20 pontos; A = 8 pontos; C = 12 pontos; I = 0 pontos. Total = 40 pontos.
- **Acumulado em AS:** R = 5 pontos; A = 0 ponto; C = 21 pontos; I = 2 pontos. Total = 28 pontos.
- **Acumulado em PR:** R = 5 pontos; A = 0 ponto; C = 15 pontos; I = 4 pontos. Total = 24 pontos.
- O total de pontos acumulados na fase de construção é de 92.

Agora observe a fase de Elaboração na figura 45:

- Acumulado em AN: R = 5 pontos; A = 4 pontos; C = 15 pontos; I = 2 pontos. Total = 26 pontos.
- Acumulado em GP: R = 10 pontos; A = 4 pontos; C = 12 pontos; I = 4 pontos. Total = 30 pontos.
- O total de pontos acumulados na fase de Construção é de 56.

Observe a seguir a análise dos dados:

- A fase de Construção (92 pontos) exige um maior esforço de serviços do que a fase de Elaboração (56 pontos).
- O Gerente de Sistema (GS = 40 pontos) é o profissional mais exigido nas atividades do projeto de software.
- Apenas para a estimativa de cálculos, se considerar o ponto como equivalente a 1 hora de serviço ou o custo de R\$ 100,00/h, temos que a atividade de Construção levará 92 H.h (Homens.hora) a um custo de R\$ 9.200,00.



Observação

O mesmo método de análise pode ser feito também por disciplina. Por exemplo: a disciplina Implementação obtém: R = 5 pontos; A = 0 ponto; C = 15 pontos; I = 6 pontos. Total = 26 pontos. Ou seja, a atividade de Implementação leva 26 H.h a um custo de R\$ 2.600,00.



Resumo

Nesta unidade, traçamos um panorama do alinhamento entre técnicas, processos e ferramentas para promover um desenvolvimento de software estruturado e eficiente.

A fusão do produto e do processo aborda como alinhar o desenvolvimento do produto ao processo. Essa integração efetiva permite que o software atenda aos requisitos técnicos e às necessidades do cliente.

Vimos também que o domínio da análise do engenheiro de software compreende aspectos como: foco na qualidade, enfatizando práticas que asseguram a qualidade durante todas as etapas do ciclo de vida do software; e o processo com base nas atividades a serem desenvolvidas, determinando estruturas bem definidas que guiam as etapas do desenvolvimento baseando-se em abordagens iterativas e incrementais.

Em uma abordagem colaborativa, o método JAD envolve stakeholders e desenvolvedores em workshops estruturados com o objetivo de definir requisitos e soluções, promovendo maior alinhamento entre as partes.

Estudamos o gerenciamento da equipe de desenvolvimento com o emprego do método MR do PMBOK, usando os atributos RACI. Estes, ao serem integrados aos métodos PERT/CPM, auxiliam no planejamento e no monitoramento de tempos e custos do projeto, mantendo a equipe alinhada com os objetivos do projeto. A MR RACI é flexível o suficiente para adaptar de forma rápida as atividades das metodologias ágeis, aumentando a produtividade de software.

Em seguida, acentuamos o RUP e suas estruturas, como fases e workflow, segmentando o ciclo de vida em concepção, elaboração, construção e transição. A progressão temporal é determinada pela estrutura dinâmica e a estrutura estática organiza os artefatos de software a serem produzidos em disciplinas e atividades, mantendo um alinhamento consistente entre as tarefas a serem desempenhadas.

Nesse contexto, foram apresentadas algumas extensões do RUP, como o Praxis, que integra práticas ágeis ao RUP, e o Iconix, que tem como foco o design orientado a objetos. Vimos também que o OpenUP é uma metodologia do EPF que apresenta uma versão simplificada e ágil para equipes menores e com princípios colaborativos.

Finalmente, destacaram-se métodos, ferramentas e técnicas com o uso da MR RACI para definir papéis claros em atividades do RUP, promovendo eficiência no desenvolvimento.



Exercícios

Questão 1. (Cesgranrio 2024, adaptada) Na gestão de projetos, é recomendável que haja um processo para identificar e analisar regularmente suas partes interessadas (stakeholders), documentando informações relevantes sobre interesses, envolvimento, interdependências, influência e impacto potencial no sucesso do projeto.

As partes interessadas de um projeto:

A) Incluem os membros da equipe do projeto, excetuando os profissionais de empresas terceirizadas ou fornecedores.

B) São as pessoas ou os grupos que fornecem recursos e suporte para o projeto, sendo responsabilizados pelo seu sucesso.

C) São as pessoas ou os grupos que necessariamente estão envolvidos com aspectos financeiros do projeto.

D) São as pessoas ou os grupos afetados ou que podem afetar o projeto apenas de forma positiva.

E) Podem ser um indivíduo, um grupo ou uma organização que possa afetar, ser afetado ou sentir-se afetado por uma decisão, uma atividade ou um resultado de um projeto, um programa ou um portfólio.

Resposta correta: alternativa E.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: as partes interessadas incluem todos os envolvidos, inclusive terceirizados e fornecedores, que, muitas vezes, são essenciais para o sucesso do projeto.

B) Alternativa incorreta.

Justificativa: embora stakeholders de fato forneçam recursos e suporte, a definição é muito restrita para diversos tipos de projetos, como os de software, nos quais há muitos outros tipos de interessados além dos financiadores.

C) Alternativa incorreta.

Justificativa: stakeholders em projetos de software vão muito além dos aspectos financeiros, incluindo usuários finais, equipe de desenvolvimento, órgãos reguladores etc.

D) Alternativa incorreta.

Justificativa: stakeholders podem afetar o projeto tanto positiva quanto negativamente. Por exemplo, em projetos de software, usuários resistentes a mudanças podem afetar o projeto negativamente.

E) Alternativa correta.

Justificativa: o texto da alternativa traz a definição mais completa e alinhada com os princípios de gestão de projetos de software, abrangendo diversos tipos possíveis de stakeholders.

Questão 2. (Cespe-Cebraspe 2024, adaptada) Determinada disciplina ou fase, pertencente à dimensão estática do RUP, visa fornecer uma estrutura para gerenciar risco, bem como equilibrar objetivos concorrentes, a fim de superar restrições para entregar com sucesso um produto que atenda às necessidades dos clientes. Trata-se da:

A) Disciplina implementação.

B) Disciplina gerenciamento de projetos.

C) Fase construção.

D) Fase transição.

E) Fase elaboração.

Resposta correta: alternativa B.

Análise da questão

No RUP, existem dois conceitos fundamentais para organizar o desenvolvimento de software: fases e disciplinas.

As fases representam a linha do tempo do projeto e dividem o ciclo de vida em etapas sequenciais com objetivos específicos. Cada fase termina com um marco que avalia se os critérios para avançar foram atendidos.

As disciplinas são áreas de atividade que ocorrem ao longo de todo o projeto, independentemente da fase. Elas representam as melhores práticas a serem aplicadas continuamente.

No RUP, o gerenciamento de projetos é uma das disciplinas centrais. Trata-se de uma disciplina que planeja e controla o projeto, gerencia riscos, balanceia requisitos concorrentes e garante que o produto entregue atenda às expectativas, considerando as restrições impostas.